

Drone Flight Controller, Project 9 - MLiS Part I

Jamie Mair
jamie.mair@nottingham.ac.uk

1 Introduction

With the rise of consumer electronics, many people have been building their own drones at home, resulting in many different methods of controlling these drones. The control of these drones is usually programmed into a flight controller. These drones require flight controllers that vary greatly in complexity, depending on the task. While most drones are programmed to respond to user controls, this task will focus on autonomous flight, focusing on getting the drone to reach a series of targets. The task is to write a drone flight controller which learns to control two motors to navigate the drone to a specified target.

1.1 Environment & Physics

To simplify the project we will only consider a drone flying in two dimensions, and therefore only having to supply two thrust values to the target.

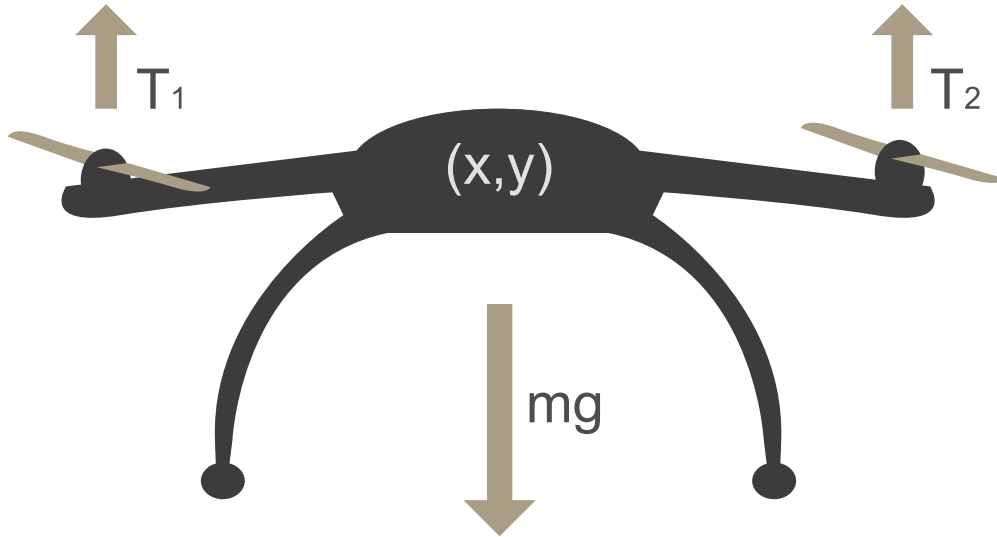


Figure 1: Diagram of the forces acting on the drone.

Considering the drone as a rigid body, with two propellers which can provide thrust, we know that if we match T_1 and T_2 , the thrust of the left and right propellers respectively (as shown in Figure 1), the drone will not rotate. However if $T_2 > T_1$ then the drone will start to rotate counterclockwise, and vice versa if $T_1 > T_2$. Additionally, gravity acts on the drone, which means that in order for the drone to stay still it must provide enough thrust to counteract the force of gravity. We can express this with the equation

$$T_1 + T_2 = mg, \tag{1}$$

where m is the mass of the drone and g is the acceleration due to gravity. For simplicity, we will consider $mg = 1N$, where the force is measured in newtons N . Each propeller is capable of producing $1N$ of force perpendicular to the plane of the drone, as shown in the diagram.

1.1.1 Example of vertical motion

In order for the drone to avoid rotating, we can set $T_1 = T_2$. We can work out the total resultant force on the drone, in the vertical axis as

$$F = T_1 + T_2 - mg. \quad (2)$$

We can know that thrust of each propeller required to keep the drone stationary is simply $\frac{1}{2}mg$. This is a useful quantity to measure the thrust relative to. If we set the thrust, T , of each propeller as

$$T = \frac{1}{2}mg + \epsilon, \quad (3)$$

then if $\epsilon > 0$ then the drone will accelerate upwards, and if $\epsilon < 0$ then the drone will accelerate downwards. This gives us one easy parameter to control the vertical altitude of the drone.

A simple heuristic¹ one could use to move the drone to a specified altitude, providing the drone is level, would be to use ϵ as

$$\epsilon = f_{\epsilon_{\min}, \epsilon_{\max}}(-k(y' - y) - b\dot{y}), \quad (4)$$

where y' is the target altitude, y is the current altitude, $\dot{y}$ is the velocity in the y direction and k and b are just positive constants. $f_{\epsilon_{\min}, \epsilon_{\max}}(x)$ simply denotes a clipping function that has $f_{\epsilon_{\min}, \epsilon_{\max}}(x) = \min(\epsilon_{\max}, \max(x, \epsilon_{\min}))$. The reason we need to include this is because the drone will usually have a maximum thrust from each rotor, which can only push in one direction. We also know the minimum value of the thrust is 0. This heuristic for ϵ will cause the drone to perform damped simple harmonic motion around the target altitude, which will eventually settle due to the drag coefficient b .

1.1.2 Example of horizontal motion

Movement laterally requires a bit more thought. In order to have any resultant force in the lateral direction, the drone needs to rotate, such that the thrust vectors are now at an angle θ with the upwards vertical. We will call this angle θ the pitch of the drone. Usually, a drone flying in 3 dimensions has 3 angles for pitch, yaw and roll, but in two dimensions we need only consider pitch. In order for us to adjust this θ , we must make T_1 and T_2 imbalanced. One choice for doing this is to perturb the equilibrium thrust T_{eq} by an amount γ , such that $T_1 = T_{\text{eq}} + \gamma$, $T_2 = T_{\text{eq}} - \gamma$. This will result in a resultant torque $\tau \propto 2\gamma$. A positive value of γ will cause the drone to begin rotating clockwise, which is the direction we will choose to define positive θ . Notice that now the resultant force in the vertical plane is $F_y = 2T_{\text{eq}} \cos \theta - mg$, and hence if we want to eliminate vertical motion we can simply set $mg = 2T_{\text{eq}} \cos \theta$ and solve for T_{eq} . Notice that now γ has to be constrained such that T_1 and T_2 neither fall below 0 or exceed a maximum thrust, which we can define as being T_{max} .

If we wish to move laterally with a constant force, we must set a constant pitch. We can do this in the exact same way as we tried to adjust y to hit a target y' . We can define a target pitch θ' and choose new constants k_θ and b_θ and use a new equation to work out a possible value for γ given as

$$\gamma = f_{\gamma_{\min}, \gamma_{\max}}(-k_\theta(\theta' - \theta) - b_\theta\dot{\theta}), \quad (5)$$

where $\dot{\theta}$ is the angular pitch velocity. We can use this for consistent control of the lateral motion by first choosing a finite pitch Θ , say $\Theta = 10^\circ$, and then setting θ' equal to $+\Theta$ when moving to the right and $-\Theta$ when moving to the left, and 0 for being stationary.

¹A heuristic can be thought of as simply a “rule of thumb” method that attempts to help solve a complex problem, but is often far from optimal.

Using this heuristic controller model for T_1 and T_2 , we can make a very simple flight controller that repeats the following algorithm to get to the coordinates (x', y') :

1. If $x' > x$ then set $\theta' = +\Theta$, if $x' < x$ then $\theta' = -\Theta$ otherwise $\theta' = 0$.
2. If θ' is non-zero, then calculate γ and use $T_1 = T_{eq} + \gamma$, $T_2 = T_{eq} - \gamma$, making sure to calculate T_{eq} so that the resultant vertical force is zero.
3. If θ' is zero, then calculate ϵ and use $T_1 = T_2 = T_{eq} + \epsilon$, where $T_{eq} = \frac{1}{2}mg$.
4. Set the thrust of the drone to the calculated value of T_1 and T_2 and iterate the simulation to the next time step.

If you were to implement this heuristic, you would see that the drone will be able to hit most targets (given enough time and sensible hyper-parameters for k , b etc...). However, it is easy to see that the drones flight is far from optimal and requires a lot of specialist knowledge of the system. This project is all about trying to find alternative methods of choosing T_1 and T_2 to make the drone hit specified targets.

2 Task

We have provided a Python implementation of a full drone simulation, given in a GitHub repository[1], which has instructions on how to get started with the code. The drone simulation only uses the *numpy* package, and you should not use any specialist machine learning packages for your project. In general you should only need *numpy* and *matplotlib*, but if you are unsure, contact a member of staff to ask. We want you to program a model which uses information contained in the provided drone object as an input, and outputs two values of thrust (i.e T_1 and T_2). The code has been simplified such that $mg = -1$ and as such the drone initially remains stationary at $(0, 0)$ when $T_1 = T_2 = 0.5$. The maximum thrust for T_1 and T_2 is 1. We have programmed the dynamics of the drone for you so that you can focus on the flight controller aspect. This dynamics includes drag forces on both the rotation and the planar motion, this should make programming the controller much easier as it lessens the impact of “over-shooting” a target.

2.1 Objectives

The key parts of this project are as follows:

- You **must** formulate the problem of “hitting targets” as a reinforcement learning problem, describing what are the **states**, **actions** and **rewards** chosen.
- You **must** discuss a way of scoring the performance of an agent (i.e. the reward function) and explain how your choice affects not only the desired behaviour of the drone, but also the training process.
- You **must** be able to describe and demonstrate an algorithm which is able to learn with experience, as characterised as increasing the expected return of your agent.

You are free to choose the type of model used to solve this problem, and the algorithm chosen for learning. Refer to the lecture notes and [2] for algorithms.

2.2 Tips

You will need to decide on how to construct your model. As an example you could implement heuristics, such as the ones described in earlier sections. These heuristics will have parameters which can be tuned via black box optimisation methods, according to the reward function chosen.

An alternative, is to program a set of heuristics which can be chosen as possible actions. For example, one can have a heuristic called *arrest motion*, which aims to choose the thrusts such that the drone comes to a stop. One can think of a few heuristics which can be used to set the thrusts to achieve certain objectives. One can then form the action space by choosing which heuristic to use to calculate the thrusts. One can formulate this as a RL problem, and use standard algorithms to optimise this.

One consideration when applying reinforcement learning techniques without access to a neural network library, is thinking about discretising the input state space to a manageable size. Another option is to construct a very shallow neural network, possibly a perceptron and implement the “REINFORCE” algorithm, along with some form of backpropagation. This is a large task and should only be considered if you are confident enough with programming. The main focus of this project should be formulating this as a RL problem and trying to apply some of the algorithms learnt.

The most important part of the project is formulating an appropriate reward function, which will accurately encode the goal of the reaching the targets, while also providing some information which can be useful for Monte-Carlo based algorithms.

An example demonstration of the drone simulation is given as a video here². In this video you can see that the drone pitches towards the right and accelerates upwards. It overshoots the target altitude and consequently has to reduce it’s net thrust to descend down to hit the target. This simulation is programmed such that once the drone hits a target, a new one is supplied in a chain. The aim is to hit as many targets as possible in a given time.

3 Python Implementation

We have provided a series of Python files[1], which gives you a starting place to experiment with building different flight controllers. Please carefully read through the code provided, and the `README` to get started.

The provided code contains a `Drone` class, which has all the information about the state of the environment. This class contains a function called `set_thrust()`, which takes in a tuple argument for a pair of thrust values for T_1 and T_2 respectively that must be real numbers between 0 and 1 inclusive; the other method is called `step_simulation()`, which when called will solve the dynamics of the flight to move the drone to the next time step. These two methods are what you will use to act as the controller. Please do not modify this file when solving your code, try to only use the methods provided and treat all the variables in the class as “read-only” when not using the functions of the class.

We have also provided a way to run the simulation visually using the “*pygame*” module. We have also provided a template class for you to implement a given flight controller. We have also provided a heuristic flight controller, similar to the one described in the introduction.

In order to get started running the code you will need to setup Python on your computer. You should be able to find resources online to get Python installed on your computer. Once installed, you will need to make sure that you have *numpy*, *matplotlib* and *pygame* installed using pip or conda (or a package manager of your choice). If you have any trouble getting the code to run using your installation, then you can create a virtual environment and use the “requirements.txt” file provided to setup an environment we have tested

²https://youtu.be/nX_FQsFuDfo

the code with.

We have provided a template class, in a file called “*custom_controller.py*” for you to write your code in. There are two other controller files for you to look at to see where to get started with training implementations.

Please remember that you should not use machine learning packages such as *tensorflow* or *pytorch* in your implementation. Packages for strictly visualisation are allowed, and other modules that do not contribute to the algorithm of your model should also be allowed, but be sure to ask staff if you are unsure.

We have chosen Python as the language of implementation here, however, if you have a lot of experience using another language and your entire group agrees to using another language contact Jamie³ to discuss this possibility. This is a possibility, since Python is not a fast language and you may run into trouble with long simulations as the code is by no means written to be quick. Changing is not advised since it will set you back in your development and will only be allowed if all members of the group agree and you can demonstrate your proficiency in a chosen alternative language.

References

- [1] Jamie Mair. *Drone Flight Controller Github Repo*. <https://github.com/JamieMair/DroneControllerMLiS>. 2020.
- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. <http://incompleteideas.net/book/the-book.html>. 2018.

³jamie.mair@nottingham.ac.uk